

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Lectures and recitations are back in-person
- Upcoming deadlines
  - Homework 2 is due next Monday at 11:59 pm
  - Lab 1 is due on 2/4 at 11:59 pm
  - Project 1 is due on 2/18 at 11:59 pm
    - Explained in this week's recitations

Previous Lecture …

Three-usage problems of Semaphores

- compromising mutual exclusion
  - **Solution: Mutex**
- deadlock
  - **Solution: Not yet discussed**
- priority inversion
  - **Solution: priority inheritance**

Muddiest Points (basic concepts)

- Deadlock
- mutual exclusion

Muddiest Points (semaphore)

- What the semaphore value means.
- Can we see a process that utilizes a semaphore/spinlock and trace it?
- Where we move after down is called

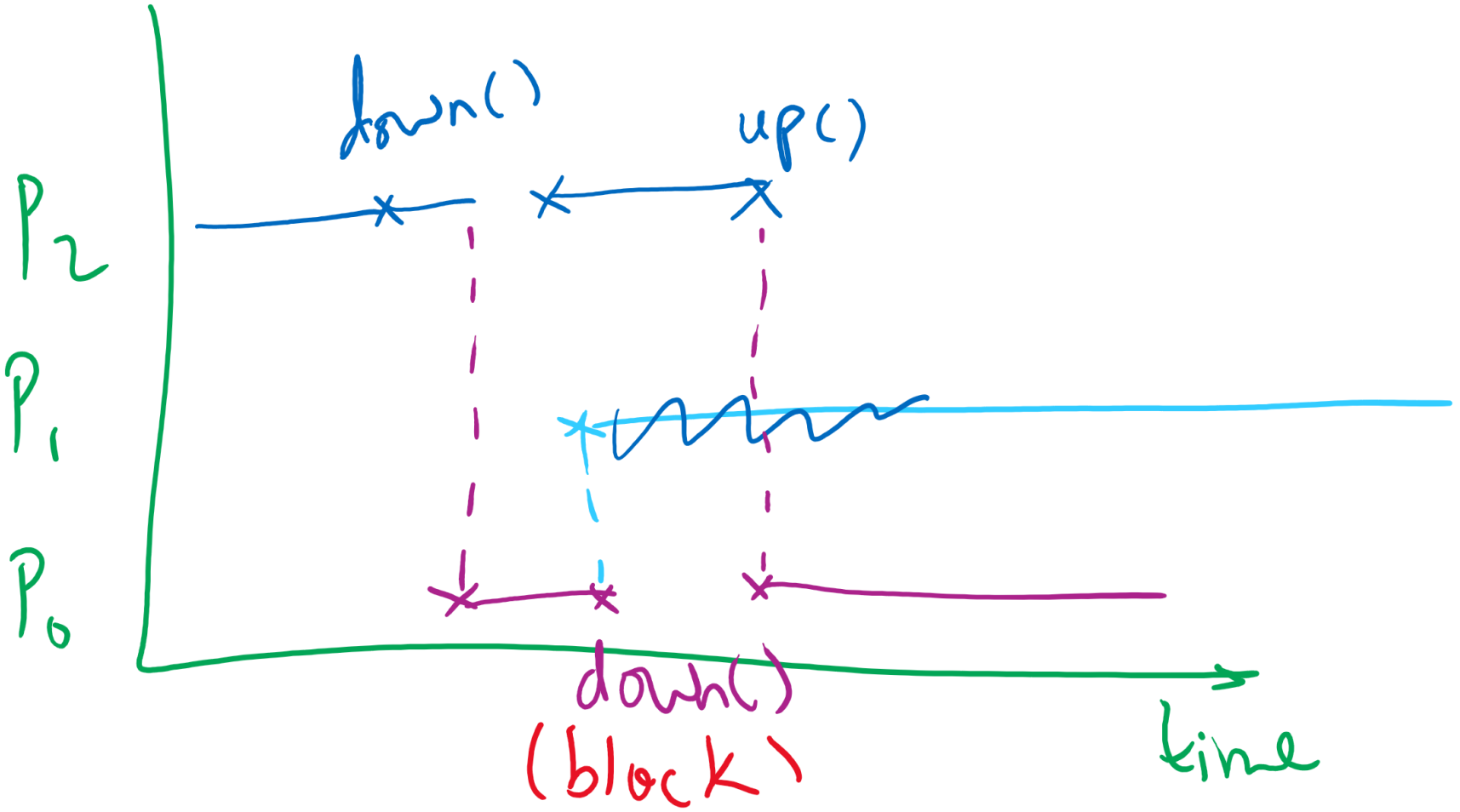
Muddiest Points (Mutex)

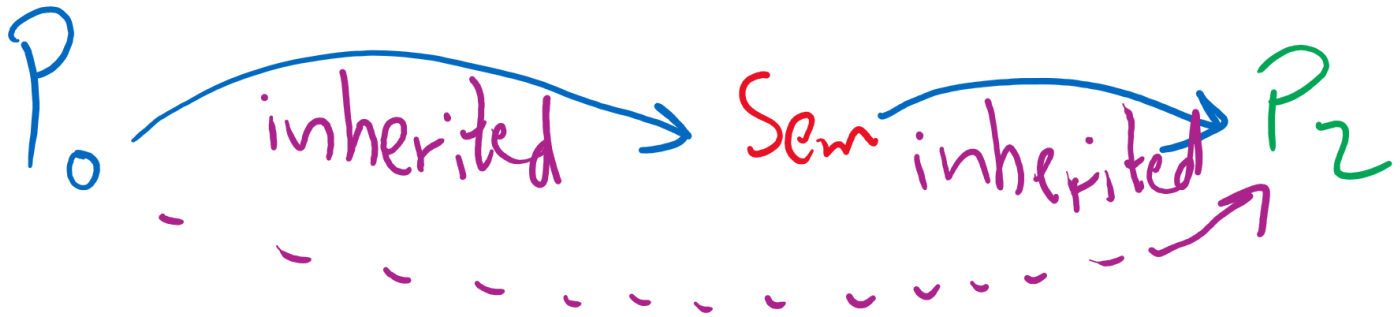
- With mutex, what happens when a process tries to up() when it hasn't down()'d, does it skip over the instruction or does it do something else?
- how does a mutex semaphore verify that the same process that called down on it in the first place is the one that called up on it?
- Can a mutex have a negative value or is it only 0 or 1?
- Advantages/disadvantages of Mutex

Muddiest Points (priority inheritance)

- What is priority inheritance?
- I'm confused on what the solution for priority inversion is or did we not mention it yet?
- Does Priority Inheritance solve the Deadlock and Starvation problems along with the Priority Inversion problem?
- Why running P1 in the priority inversion question not down the semaphore?
- Does priority inversion (of semaphores) cause the lower process to run instead of the higher priority process indefinitely or does this resolve itself? If so how?

Semaphore Usage Problem: Priority Inversion

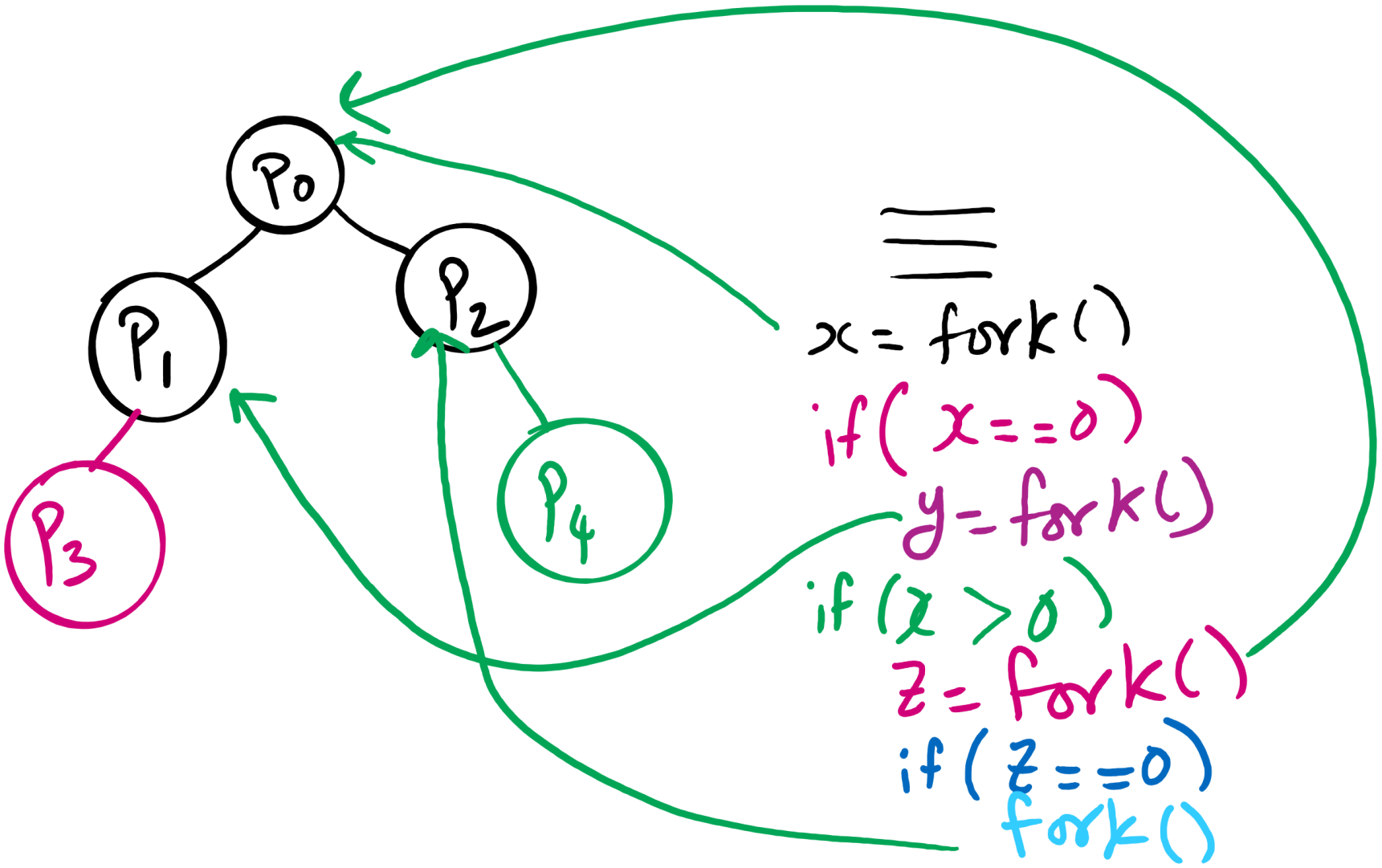




Muddiest Points (forking 1)

- if fork creates copies of the process that called it, how can you tell which copy has priority to run first?
- child processes & where they pick up in the code
- Not exactly sure how semaphores are inherited between processes
- In the last example, when p1 calls fork and creates the child p3, will p1's PCB be copied to p3, which means the content of p0's pcb will be in p3 too?
- What is a PID and how is it related to the return value of fork()?
- P3 is forked and created before P2 so why is it named P2? Is it simply renamed after P1 gets forked again?
- Is the order of possible outcomes from a forking program the permutation of any of the leaf nodes from a forking tree?

fork()'s of fork()'s



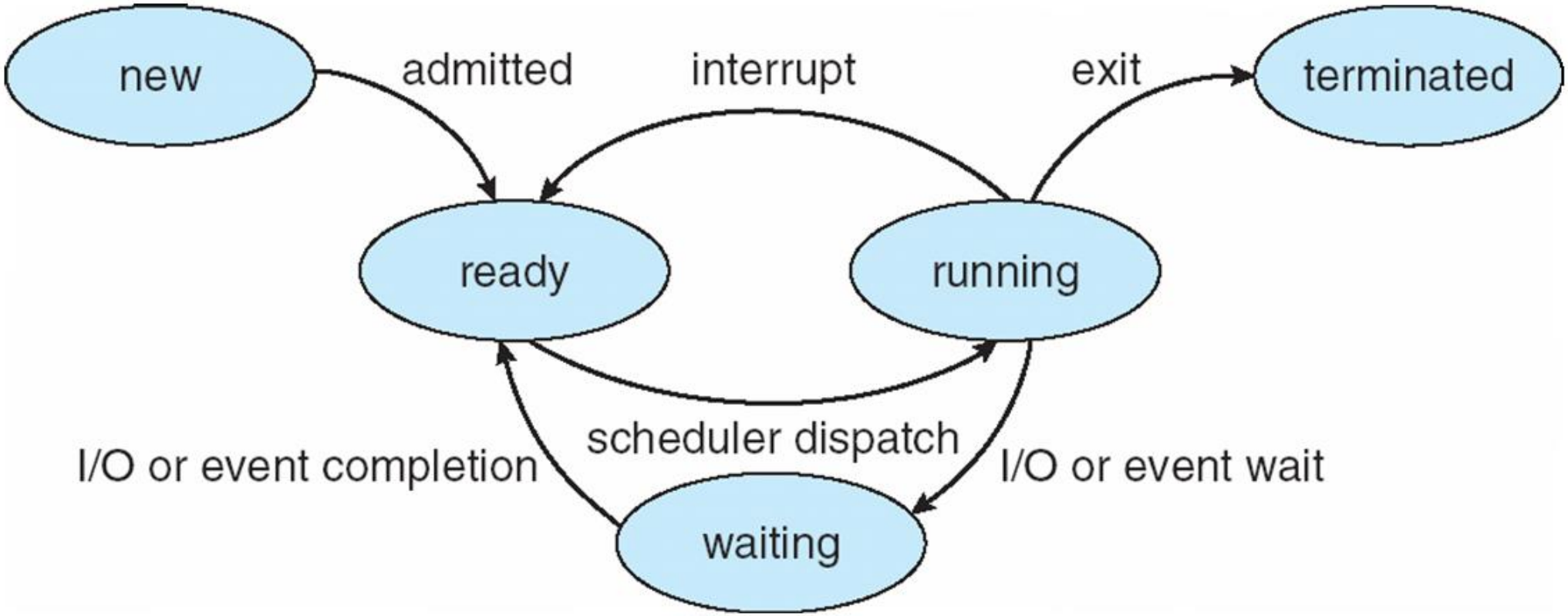
Muddiest Points (forking 2)

- does the fork return zero for all children?
- Somewhat confused about what order the various forks will actually form. I see why and from where they form, but not how it decides which one to run
- how are all processes a fork of another process? i know the root of the tree is another process in the OS, but i don't get how for example a program you create and execute is a fork of something else.
- the forking process with child processes. The relationship between the number of child processes created and the number of times fork() is called by the parent process
- what int fork returns for the child and parent process
- practical uses of forking?

Today's Problem/Question

How are processes created and terminated?

Process Lifecycle (AKA Process States)



Process Creation

- Via fork() syscall
- Parent process: the process that calls fork()
- Child process: the process that gets created
- Memory of parent process copied to child process
  - Too much copying
  - Even not necessary sometimes
    - e.g., fork() followed by exec() to run a different program
  - Optimization trick:
    - copy-on-write

fork() example

```c
int main(){
  int a, b, c, x, y, z;
  printf("Start\n");
  x = fork();
  y = fork();
  if(x>0){
    z = fork();

  } else {
    a = fork();
  }
  if(z > 0 && a ==0){
   b = fork();
  }
  fork();
  printf("End\n");
  return 0;
}
```

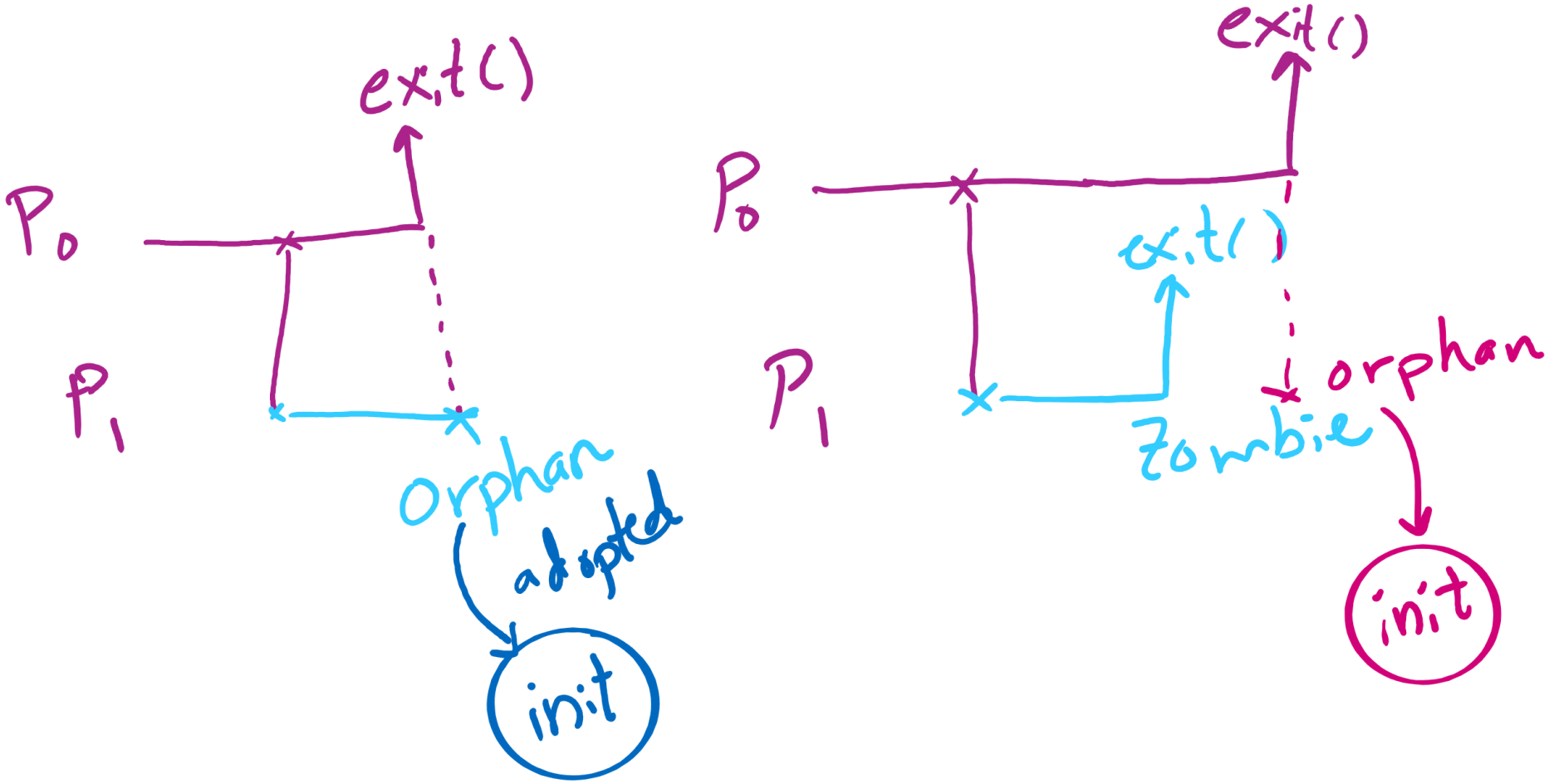
Process Termination

- Via **`exit`**(), **`abort`**(), or **`kill()`** syscalls
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- When a process terminates
  - If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
  - If parent terminated without invoking **`wait`**, process is an **orphan**
    - adopted by the **`init`** process

Orphan vs. Zombie Processes



Benefits of Orphan Processes

- Allow a long-running job to continue running even after session (e.g., ssh connection) ends.
  - The nohup command does that
- Create daemon processes
  - Long-running background processes adopted by the init process.

Thread Synchronization

Synchronization issues apply to *threads* as well

- Threads can share data easily (same address space)
- Other two issues apply to threads

Process vs. Thread

